

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



Guia Ejecucion Addino

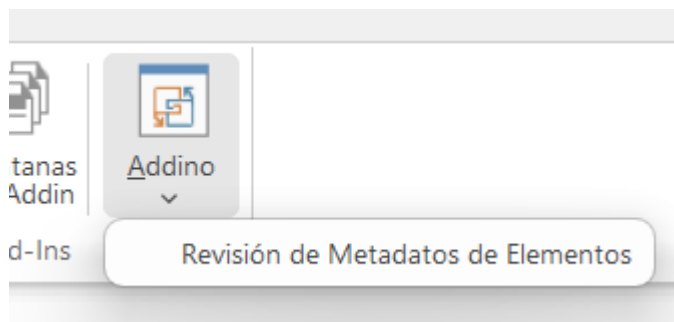


Ezequiel Pino

Inicio de Desarrollo	3
Primer Add-in de prueba	3
Desarrollo asistido por Ecosistema Gentle-AI	4
Tecnologías utilizadas	5
3. Estructura principal del proyecto	6
AddinoClass.cs	6
MetadataElementRow.cs	6
MetadataReviewForm.cs	6
Instalación Addino	7
Requisitos previos	7
1. Abrir la solución	7
2. Compilar Addino	7
3. Registro del Add-in en Enterprise Architect	8
4. Abrir Enterprise Architect	9
5. Seleccionar un paquete	9
6. Abrir Addino	10
7. Elementos mostrados	11
8. Columnas de la grilla	12
9. Editar elementos	12
10. Edición multilínea de Notas	13
11. Guardar cambios	13
12. Guardar sin modificaciones	14
13. Verificar que un cambio fue guardado	14
14. Cancelar cambios	15
15. Cambios guardados y cambios pendientes	16
16. Manejo de errores	16
17. Evidencia y Pruebas manuales realizadas	17
18. Uso de Inteligencia Artificial	17
19. Funcionalidades opcionales no implementadas	18

Guia de Ejecución **Addino** – Revisión de Metadatos para **Enterprise Architect**

Addino es un Add-in desarrollado en C# para Sparx Enterprise Architect como parte del desafío técnico de práctica de Proagile. El nombre nace de combinar **Add-in + Pino**, mi apellido: **Add-in + Pino = Addino**



El objetivo principal de Addino es facilitar la revisión y actualización de los metadatos de los elementos contenidos dentro de un paquete de Enterprise Architect. En la fase inicial, que cumple con el alcance mínimo exigido por el desafío, incluye:

Desde una única grilla es posible visualizar los elementos directos de un paquete y modificar:

- **Nombre**
- **Alias**
- **Notas**

Mientras que los siguientes campos se muestran únicamente como información:

- **Tipo**
- **Estereotipo**

Los cambios se mantienen localmente mientras se trabaja sobre la ventana y solamente se escriben en el repositorio cuando se presiona **Guardar**.

Inicio de Desarrollo

Antes de implementar la solución definitiva, decidí familiarizarme con el mecanismo de Add-ins de Enterprise Architect.

Primer Add-in de prueba

Como primer paso seguí la guía de creación de Add-ins proporcionada para el desafío.

Creé un proyecto básico en Visual Studio utilizando:

- C#
- .NET Framework
- Interop.EA
- Registro COM
- Enterprise Architect

La primera versión implementaba únicamente un menú de prueba, como el de la guía, con acciones simples como:

- Say Hello
- Say Goodbye

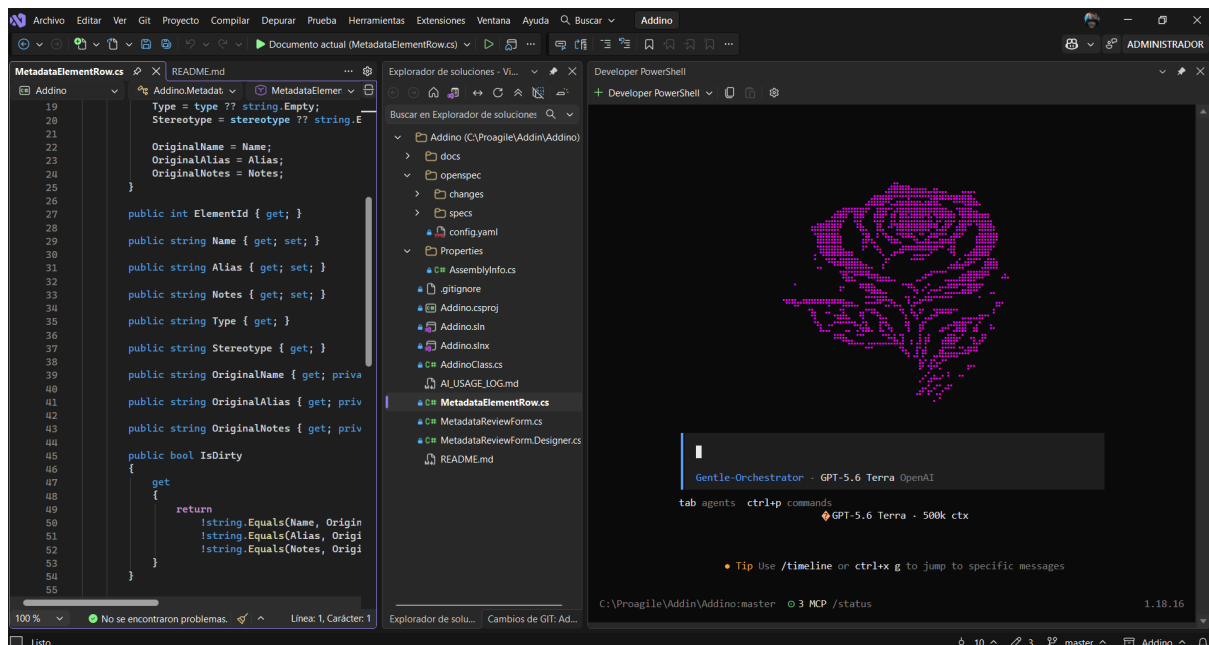
Esta implementación me permitió comprobar previamente que:

- Enterprise Architect podía cargar correctamente mi ensamblado.
- La clase COM estaba registrada.
- Los callbacks requeridos por EA eran ejecutados.
- El Add-in aparecía correctamente en el menú de extensiones.
- Visual Studio podía compilar y registrar el proyecto en mi entorno.

Una vez validado este entorno base, utilicé ese mismo proyecto como punto de partida para desarrollar Addino.

Desarrollo asistido por Ecosistema Gentle-AI

Para organizar el desarrollo utilicé **Gentle AI**, integrado en mi entorno de trabajo con OpenCode, en su forma de TUI. Ejecutado directamente en la terminal de Visual Studio.



En lugar de solicitar directamente la generación completa de la solución, utilicé un enfoque de desarrollo guiado por especificaciones (**SDD — Spec Driven Development**).

El trabajo se dividió en distintas etapas:

1. Exploración del proyecto y del desafío.
2. Definición de alcance.
3. Especificación de requisitos.
4. Diseño técnico.
5. División del trabajo en tareas.
6. Implementación por unidades pequeñas.
7. Verificación y corrección de problemas encontrados.

El objetivo de este enfoque fue mantener trazabilidad entre:

Consigna → Requisitos → Diseño → Tareas → Código → Pruebas

Durante la implementación también se realizaron correcciones a partir de pruebas reales. Por ejemplo:

- configuración correcta de edición multilínea para **Notas**;
- comportamiento de la tecla **Enter** dentro de Notas;
- cierre seguro del formulario mediante **Escape**.

El detalle de las interacciones con herramientas y modelos de IA se encuentra documentado por separado en el **Registro de Uso de Inteligencia Artificial**.

Tecnologías utilizadas

La solución fue desarrollada utilizando:

Tecnología	Uso
C#	Lenguaje principal
.NET Framework 4.7.2	Framework de ejecución
Windows Forms	Interfaz gráfica
Interop.EA	Integración con Enterprise Architect
COM	Registro y carga del Add-in
Visual Studio	Desarrollo y compilación
Enterprise Architect 17.1 x64	Entorno utilizado para las pruebas

Git	Control de versiones
Gentle AI / OpenCode	Orquestación y asistencia durante el desarrollo

La solución fue validada específicamente utilizando **Enterprise Architect 17.1 x64**.

La edición Trial de Enterprise Architect no constituye un requisito propio de Addino.

3. Estructura principal del proyecto

Los archivos principales son:

```
Addino/
├── docs
├── openspec
├── Addino.sln
├── Addino.csproj
├── AddinoClass.cs
├── MetadataElementRow.cs
├── MetadataReviewForm.cs
├── MetadataReviewForm.Designer.cs
├── README.md
├── AI_USAGE_LOG.md
├── Properties/
└── AssemblyInfo.cs
```

AddinoClass.cs

Contiene la clase principal cargada por Enterprise Architect y los callbacks del Add-in:

- EA_Connect
- EA_GetMenuItems
- EA_GetMenuState
- EA_MenuClick
- EA_Disconnect

También valida que el usuario haya seleccionado un paquete válido y carga sus elementos directos.

MetadataElementRow.cs

Representa localmente cada elemento mostrado en la grilla.

Los valores editables se mantienen en memoria y no están enlazados directamente con objetos COM de Enterprise Architect.

Esto permite cancelar una edición sin modificar accidentalmente el repositorio.

MetadataReviewForm.cs

Implementa la ventana principal de revisión de metadatos:

- grilla;
- edición;
- cancelación;
- guardado;
- manejo de errores;
- persistencia mediante `Element.Update()`.

Instalación Addino

Se incluye la guía de instalación en este documento, pero también se adjunta un archivo dedicado exclusivamente a la instalación, llamado “Pino_Guia_Instalacion_Addino”, donde se detalla la instalación, incluyendo ya los objetivos opcionales del desafío.

Requisitos previos

Para ejecutar Addino se necesita:

- Windows x64.
- Visual Studio con soporte para .NET Framework.
- .NET Framework 4.7.2.
- Sparx Enterprise Architect instalado.
- Acceso a `Interop.EA.dll`.
- Permisos necesarios para compilar y registrar el componente COM.

El proyecto fue desarrollado y probado utilizando **Enterprise Architect 17.1 x64**.

1. Abrir la solución

Abrir el archivo:

Addino.sln

desde Visual Studio.

Se recomienda ejecutar **Visual Studio como Administrador**, ya que la configuración de desarrollo utilizada registra el ensamblado COM durante la compilación.

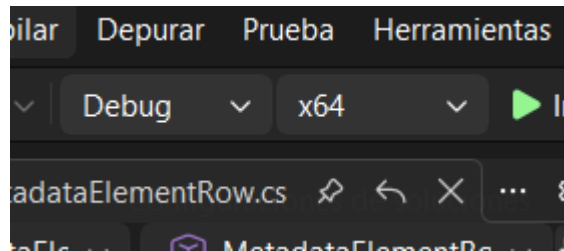
2.Compilar Addino

La configuración utilizada para las pruebas es:

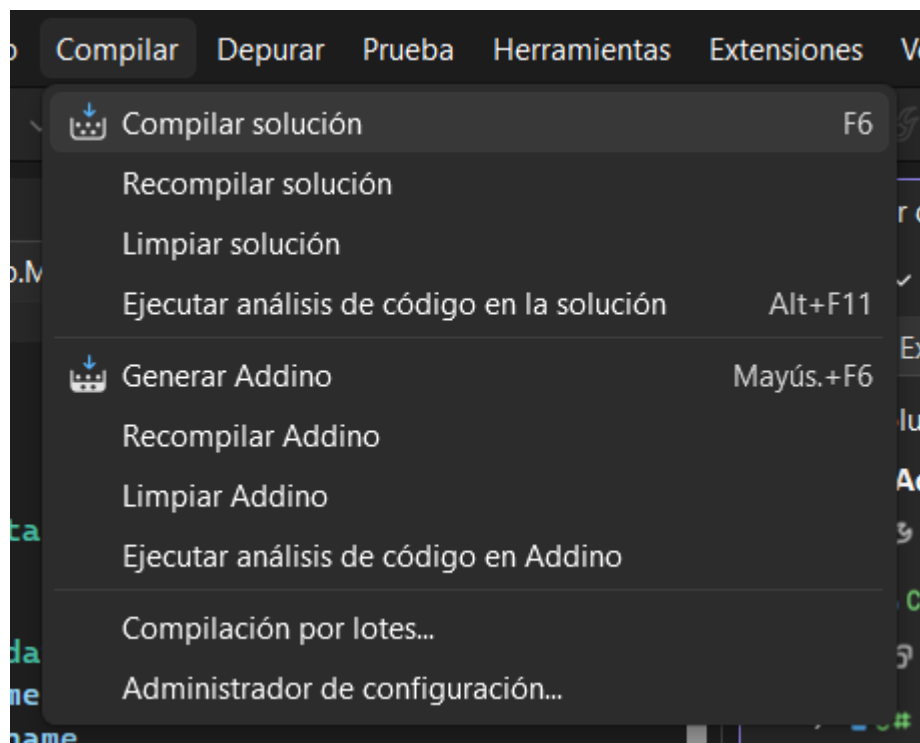
Debug | x64

Desde Visual Studio:

1. Seleccionar la configuración **Debug**.
2. Seleccionar plataforma **x64**.



3. Ejecutar **Build** → **Build Solution** (o **Compilar**→**Compilar Solución**)



También puede compilarse mediante MSBuild:

```
msbuild Addino.csproj /t:Build /p:Configuration=Debug /p:Platform=x64
```

Una compilación correcta debe finalizar sin errores.

Durante las últimas verificaciones del proyecto se obtuvo:

0 errores

0 advertencias

3. Registro del Add-in en Enterprise Architect

Enterprise Architect debe conocer la clase COM que implementa el Add-in.

En el entorno utilizado para el desarrollo, el registro se realiza bajo:

HKEY_CURRENT_USER\SOFTWARE\Sparx Systems\EAAddins64

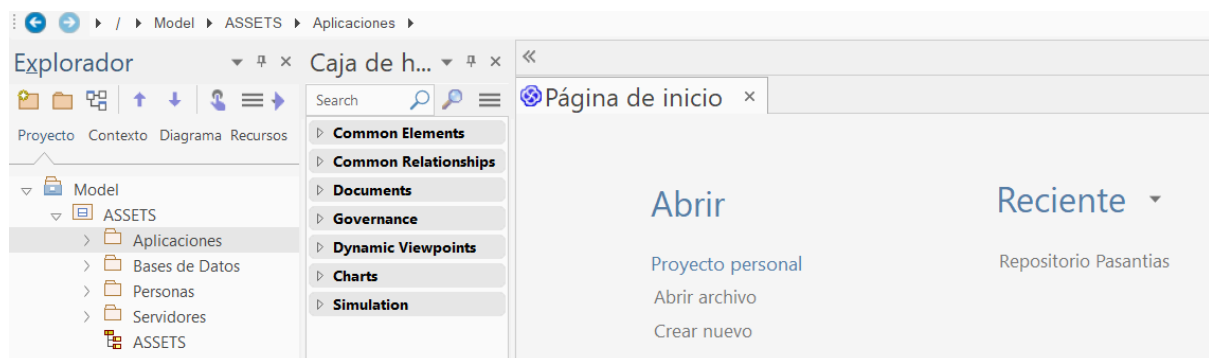
La entrada correspondiente a Addino debe apuntar a:

Addino.AddinoClass

Si el registro es correcto, Enterprise Architect podrá cargar el Add-in al iniciarse.

4. Abrir Enterprise Architect

1. Iniciar Enterprise Architect.
2. Abrir el repositorio sobre el cual se desea trabajar.
3. Esperar a que EA termine de cargar el proyecto.
4. Localizar el **Project Browser**.
5. Navegar hasta el paquete cuyos elementos se quieran revisar.



5. Seleccionar un paquete

Addino trabaja tomando como contexto el paquete seleccionado en el **Project Browser**.

Por lo tanto, antes de ejecutar la herramienta se debe seleccionar un objeto de tipo:

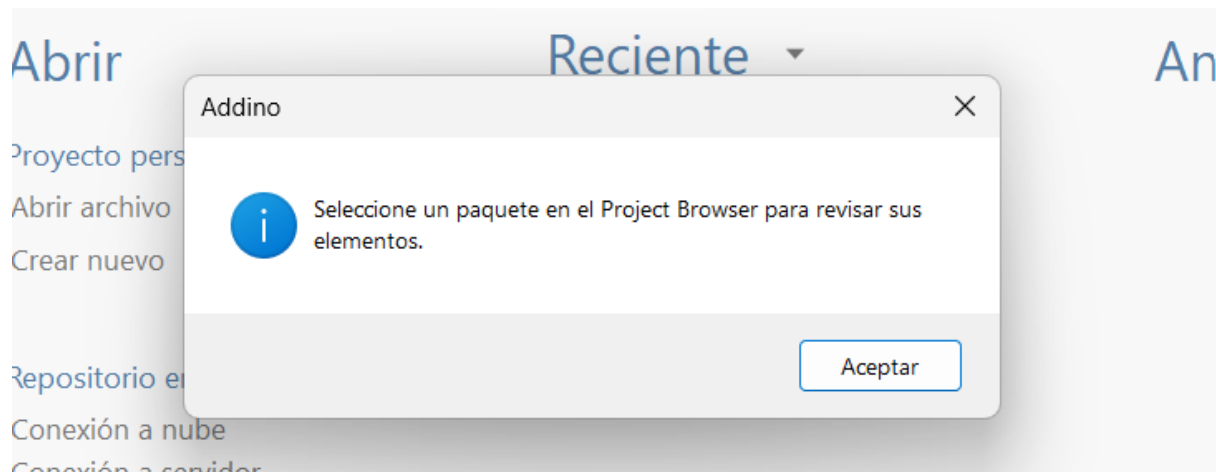
EA.Package

No debe seleccionarse:

- un elemento;

- un diagrama;
- otro tipo de objeto del modelo.

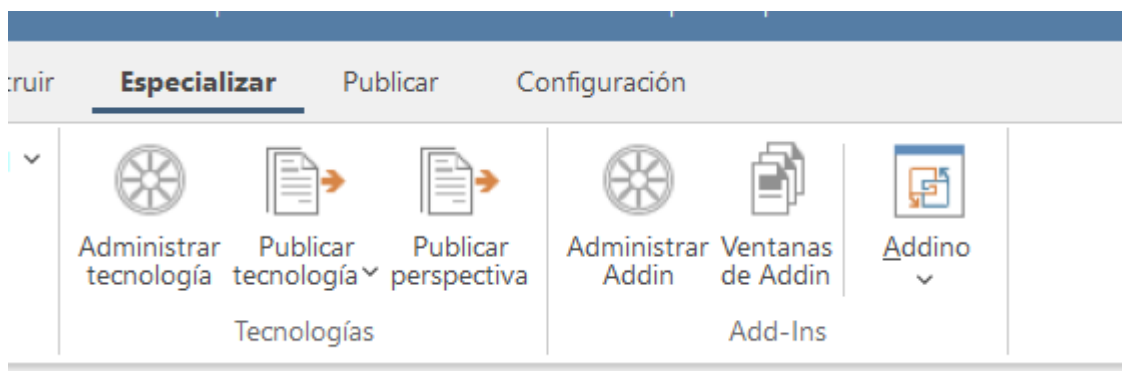
Si la selección no es válida, Addino muestra un mensaje y detiene la operación de forma segura.



6. Abrir Addino

Con un paquete seleccionado:

1. Abrir el menú **Especializar** de Enterprise Architect.
2. Buscar **Addino**.

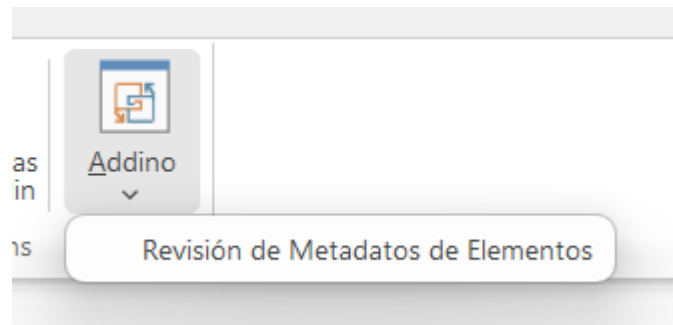


3. Seleccionar:

Revisión de Metadatos de Elementos

Se abrirá la ventana:

Revisión de Metadatos



La ventana es modal, por lo que debe cerrarse antes de continuar trabajando normalmente con otras partes de EA.

7. Elementos mostrados

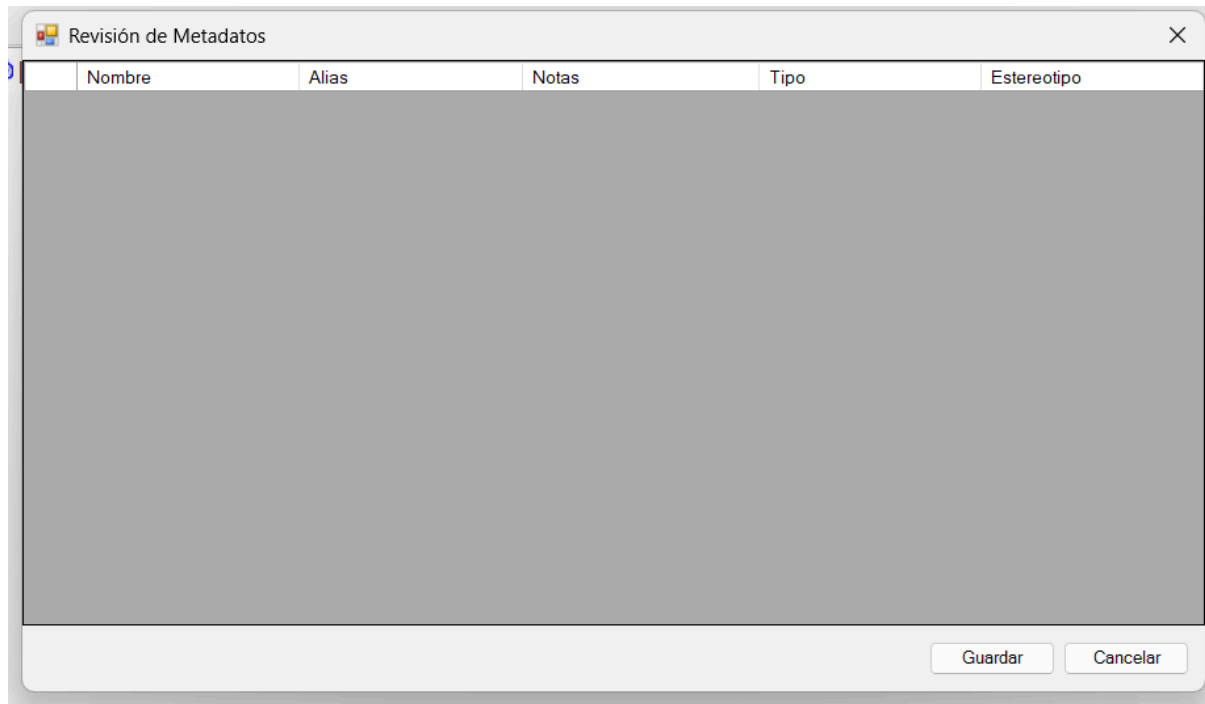
La grilla carga automáticamente los elementos contenidos **directamente** dentro del paquete seleccionado.

No se recorren los elementos contenidos dentro de subpaquetes en la fase inicial.

	Nombre	Alias	Notas	Tipo	Estereotipo
▶	Aplicación 1		Linea 1	Class	ArchiMate_ApplicationC...
	Aplicación 10			Class	ArchiMate_ApplicationC...
	Aplicación 11			Class	ArchiMate_ApplicationC...
	Aplicación 12			Class	ArchiMate_ApplicationC...
	Aplicación 13			Class	ArchiMate_ApplicationC...
	Aplicación 14			Class	ArchiMate_ApplicationC...
	Aplicación 15			Class	ArchiMate_ApplicationC...
	Aplicación 16			Class	ArchiMate_ApplicationC...
	Aplicación 17			Class	ArchiMate_ApplicationC...
	Aplicación 18			Class	ArchiMate_ApplicationC...
	Aplicación 19			Class	ArchiMate_ApplicationC...
	Aplicación 2			Class	ArchiMate_ApplicationC...
	Aplicación 20			Class	ArchiMate_ApplicationC...
	Aplicación 21			Class	ArchiMate_ApplicationC...
	Aplicación 22			Class	ArchiMate_ApplicationC...
	Aplicación 23			Class	ArchiMate_ApplicationC...
	Aplicación 24			Class	ArchiMate_ApplicationC...
	Aplicación 25			Class	ArchiMate_ApplicationC...
	Aplicación 26			Class	ArchiMate_ApplicationC...
	Aplicación 27			Class	ArchiMate_ApplicationC...

Guardar Cancelar

Si el paquete está vacío, Addino abre normalmente y muestra una grilla vacía sin generar ningún error.



8. Columnas de la grilla

La ventana muestra cinco columnas:

Columna	Editable	Descripción
Nombre	Sí	Nombre principal del elemento
Alias	Sí	Alias o nombre alternativo
Notas	Sí	Descripción del elemento
Tipo	No	Tipo de elemento
Estereotipo	No	Estereotipo aplicado

Las columnas **Tipo** y **Estereotipo** son únicamente informativas.

9. Editar elementos

Para editar un elemento:

1. Seleccionar una celda de **Nombre**, **Alias** o **Notas**.
2. Escribir el nuevo valor.
3. Continuar editando otras filas si es necesario.

	Nombre	Alias	Notas	Tipo	Estereotipo
▶	Aplicación 1			Class	ArchiMate_ApplicationC...
	Aplicación 10			Class	ArchiMate_ApplicationC...
	Aplicación 11			Class	ArchiMate_ApplicationC...
	Aplicación 12			Class	ArchiMate_ApplicationC...
	Aplicación 13			Class	ArchiMate_ApplicationC...
	Aplicación 14			Class	ArchiMate_ApplicationC...
	Aplicación 15			Class	ArchiMate_ApplicationC...
	Aplicación 16			Class	ArchiMate_ApplicationC...
	Aplicación 17			Class	ArchiMate_ApplicationC...
	Aplicación 18			Class	ArchiMate_ApplicationC...
	Aplicación 19			Class	ArchiMate_ApplicationC...
	Aplicación 2			Class	ArchiMate_ApplicationC...
	Aplicación 20			Class	ArchiMate_ApplicationC...
	Aplicación 21			Class	ArchiMate_ApplicationC...
	Aplicación 22			Class	ArchiMate_ApplicationC...
	Aplicación 23			Class	ArchiMate_ApplicationC...
	Aplicación 24			Class	ArchiMate_ApplicationC...
	Aplicación 25			Class	ArchiMate_ApplicationC...
	Aplicación 26			Class	ArchiMate_ApplicationC...
	Aplicación 27			Class	ArchiMate_ApplicationC...

Guardar Cancelar

Mientras no se presione **Guardar**, los cambios permanecen únicamente en la memoria del formulario.

El repositorio de Enterprise Architect todavía no se modifica.

10. Edición multilínea de Notas

La columna **Notas** admite texto multilínea.

Dentro de una celda de Notas:

1. Escribir una línea.
2. Presionar **Shift+Enter**.

	Nombre	Alias	Notas	Tipo	Estereotipo
✎	Aplicación 1		Hola Buenas tardes	Class	ArchiMate_ApplicationC...
	Aplicación 10			Class	ArchiMate_ApplicationC...
	Aplicación 11			Class	ArchiMate_ApplicationC...
	Aplicación 12			Class	ArchiMate_ApplicationC...
	Aplicación 13			Class	ArchiMate_ApplicationC...
	Aplicación 14			Class	ArchiMate_ApplicationC...
	Aplicación 15			Class	ArchiMate_ApplicationC...
	Aplicación 16			Class	ArchiMate_ApplicationC...
	Aplicación 17			Class	ArchiMate_ApplicationC...
	Aplicación 18			Class	ArchiMate_ApplicationC...
	Aplicación 19			Class	ArchiMate_ApplicationC...
	Aplicación 2			Class	ArchiMate_ApplicationC...
	Aplicación 20			Class	ArchiMate_ApplicationC...
	Aplicación 21			Class	ArchiMate_ApplicationC...
	Aplicación 22			Class	ArchiMate_ApplicationC...
	Aplicación 23			Class	ArchiMate_ApplicationC...
	Aplicación 24			Class	ArchiMate_ApplicationC...
	Aplicación 25			Class	ArchiMate_ApplicationC...
	Aplicación 26			Class	ArchiMate_ApplicationC...

Guardar Cancelar

3. Continuar escribiendo en la siguiente línea.

La tecla Enter inserta el salto de línea y **no ejecuta el guardado**.

Este comportamiento fue validado manualmente dentro de Enterprise Architect.

11. Guardar cambios

Cuando se presiona **Guardar**, Addino:

1. finaliza la edición activa de la celda;
2. identifica qué filas fueron realmente modificadas;
3. recupera el elemento correspondiente desde Enterprise Architect utilizando su `ElementId`;
4. actualiza únicamente:
 - Nombre;
 - Alias;
 - Notas;
5. ejecuta `Element.Update()`;
6. comprueba el resultado retornado por EA;
7. procesa las demás filas aunque alguna produzca un error;
8. muestra un resumen final en español.

Los elementos que se guardaron correctamente dejan de considerarse pendientes.

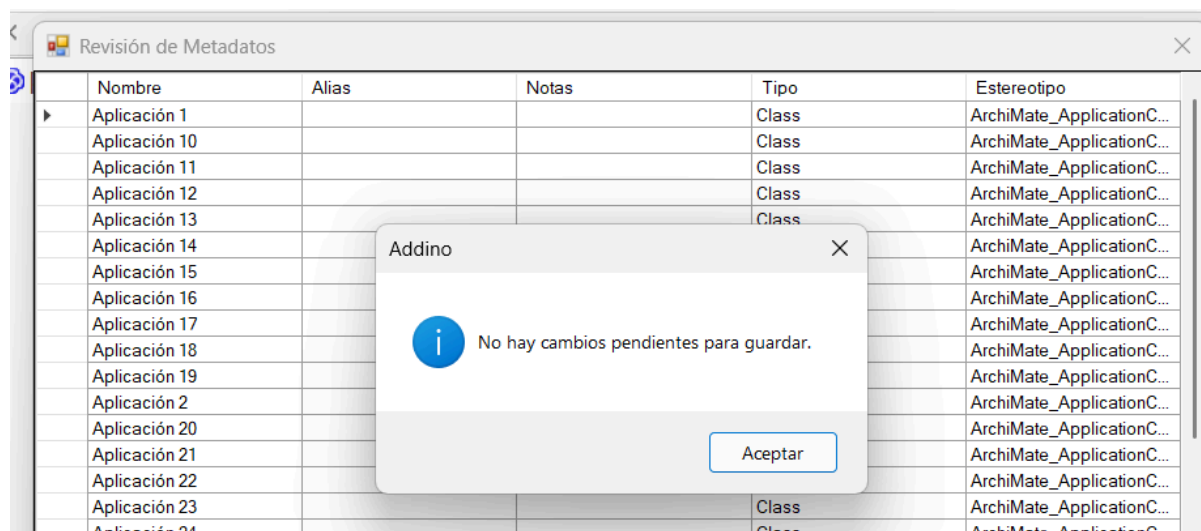
12. Guardar sin modificaciones

Si se presiona **Guardar** sin haber modificado ninguna fila, Addino no realiza escrituras innecesarias.

Muestra el mensaje:

No hay cambios pendientes para guardar.

Este escenario fue comprobado manualmente.



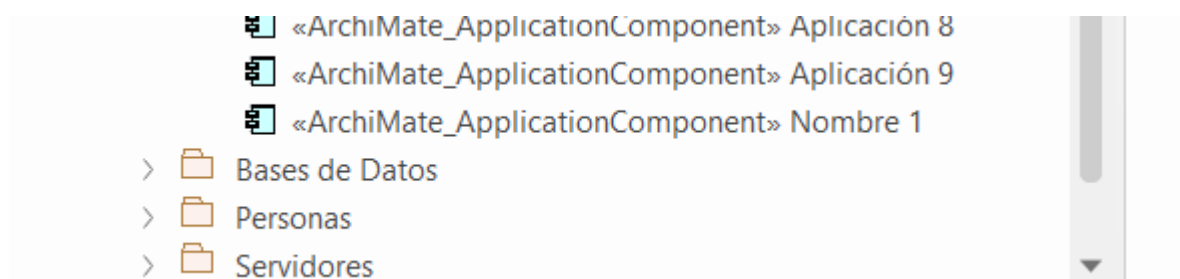
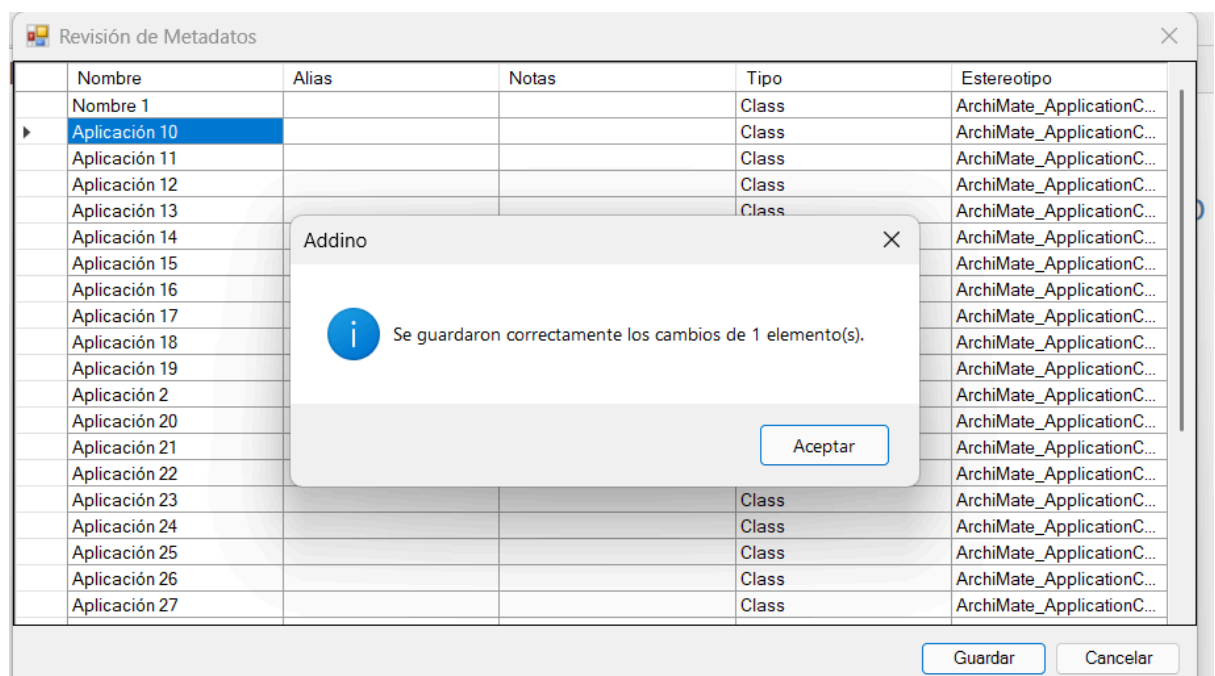
13. Verificar que un cambio fue guardado

Después de guardar:

1. cerrar la ventana de Addino;
2. localizar el elemento modificado en Enterprise Architect;
3. abrir sus propiedades;
4. revisar Nombre, Alias o Notas;
5. verificar que el nuevo valor aparezca correctamente.

También es posible volver a abrir Addino sobre el mismo paquete y comprobar que la grilla carga el valor persistido.

Durante las pruebas realizadas, los valores guardados se reflejaron correctamente en el repositorio.



14. Cancelar cambios

Una edición puede descartarse de tres maneras:

- botón **Cancelar**;
- tecla **Escape**;
- botón **X** de la ventana.

En los tres casos:

- el formulario se cierra;
- los cambios todavía no guardados se descartan;
- no se ejecuta `Element.Update()`;
- los valores previamente guardados permanecen intactos.

15. Cambios guardados y cambios pendientes

Existe una diferencia importante entre un cambio ya guardado y otro que todavía se encuentra pendiente.

Ejemplo:

1. modificar una Nota;
2. presionar **Guardar**;
3. modificar nuevamente esa Nota;
4. cerrar mediante Cancelar, Escape o X.

Al volver a abrir Addino se mantiene el valor del **último guardado**, mientras que la segunda modificación se descarta.

16. Manejo de errores

El guardado se realiza de forma independiente por cada elemento.

Si uno de ellos falla:

- Addino registra el error;
- identifica la fila o elemento afectado;
- continúa intentando guardar las demás filas.

La implementación contempla:

- `Element.Update() == false`;
- errores al recuperar un elemento;
- excepciones de Enterprise Architect / COM;
- Elementos bloqueados o no escribibles.

Estos caminos se encuentran implementados y controlados en el código.

17. Evidencia y Pruebas manuales realizadas

Durante el desarrollo realicé cada prueba de forma manual. Las mismas se encuentran detalladas en el documento “Pino_Evidencias_Pruebas_Funcionales_Addino.pdf”

Como parte de la entrega se incluye documentación adicional con:

- capturas de pantalla;
- resultados esperados;
- resultados obtenidos;
- pruebas realizadas;
- enlaces a videos completos de ejecución

También se incluye un video mostrando el flujo completo dentro de Enterprise Architect.

LINK VIDEO

18. Uso de Inteligencia Artificial

La Inteligencia Artificial fue utilizada como herramienta de apoyo durante distintas etapas del desarrollo.

En particular, Gentle AI permitió organizar el trabajo mediante agentes especializados para:

- exploración del proyecto;
- análisis de requisitos;
- diseño;
- generación de especificaciones;
- descomposición en tareas;
- implementación;
- revisión;
- corrección de problemas encontrados durante las pruebas.

Las decisiones importantes se contrastaron con:

- la consigna original;
- el código existente;
- compilaciones reales;
- pruebas manuales dentro de Enterprise Architect.

La IA no reemplazó las pruebas de funcionamiento: cada comportamiento principal fue comprobado manualmente dentro de EA.

El detalle de modelos, prompts, decisiones y evidencias se encuentra en:

Pino_Registro_Uso_IA_Addino.pdf

y en su versión fuente:

AI_USAGE_LOG.md

19. Funcionalidades opcionales no implementadas

Esta versión se concentra primero en cumplir completamente los requisitos obligatorios del ejercicio.

Por ese motivo no se implementaron los desafíos opcionales:

- recursividad en subpaquetes;
- indicador visual de filas modificadas;
- validación que impida Nombre vacío;
- botón de recarga;
- creación de nuevos elementos.

Esto mantiene el alcance de Addino enfocado en la funcionalidad requerida.